

Geomerking

Case 3: Naturhendelser

Emnekode: IN2000 – Software Engineering med prosjektarbeid

Team-nummer: 20

Teammedlemmer: David Hovde (davidhov), Jeryos Abdo (jeryosa),
Matilda Wold Dahl (matildad), Philip Elias Fleischer (philipef),
Veronica Corsepilus Melen (veroncm), Victoria Reinseth-Tollefsen
(victre)

Veiledere: Dimitrii Soltaganov og Sigurd Nordbye

Antall ord: 6264

Dato: 15.05.2026



Innholdsfortegnelse

1	Presentasjon	4
1.1	Introduksjon/sammendrag	4
1.2	Presentasjon av caset	4
1.3	Kort beskrivelse av løsningen.....	4
1.4	Beskrivelse av teamet	4
2	Brukerdokumentasjon.....	5
2.1	Applikasjonens funksjonalitet	5
2.2	Målgruppe	6
2.3	Plattformer og aksessering.....	7
3	Produktdokumentasjon.....	7
3.1	Kvalitetssegenskapene ved appen.....	7
3.1.1	Brukskvalitet	7
3.1.1.1	WCAG - Web Content Accessibility Guidelines.....	7
3.1.1.2	SavedState	8
3.1.2	Respons.....	9
3.1.3	Vedlikeholdbarhet og pålitelighet.....	9
3.2	Brukerundersøkelser og metoder for datainnsamling og analyse.....	10
3.3	Brukte API	11
3.3.1	Meteorologisk Institutt	11
3.3.2	Andre API-er.....	11
3.4	GeoScore-algoritmen.....	12
3.4.1	Beskrivelse av algoritmen	12
3.4.2	Del-scorer.....	12
3.4.3	Begrunnelse for valg av naturhendelser	13
3.5	Databasen.....	14
3.6	Kart	15
4	Prosessdokumentasjon	16
4.1	Utviklingsmetode og bruk av smidige praksiser	16
4.2	Prosjektets gang	16

4.2.1	Kick-off og valg av case	16
4.2.2	Oppstartsfasen/Sprint 0.....	17
4.2.3	Iterasjon 1	18
4.2.4	Iterasjon 2	19
4.2.5	Iterasjon 3	20
4.2.6	Sluttfasen	21
4.3	Endringer i kravspesifikasjoner – hva og hvorfor.....	22
4.4	Testing	23
4.4.1	Testing på enhet/unit testing.....	23
4.4.2	Brukertester	23
5	Refleksjon	24
5.1	Jobbe som tverrfaglig team	24
5.2	Smidig prosess og effekten av det	24
5.3	Sentrale utfordringer i utviklingsprosessen	25
5.3.1	Utfordringer med Github	25
5.3.2	Tester	25
5.3.3	Teknisk gjeld	26
6	Avslutning	26
6.1	Det viktigste vi tar med oss videre	26
6.2	Tips til neste års studenter.....	27
7	Referanseliste	28
8	Vedlegg	30
8.1	Vedlegg 1 – Samtykkeskjemaer	30
8.2	Vedlegg 2 – TeamAvtalen	32
8.3	Vedlegg 3 – Retrospektiv 1	34
8.4	Vedlegg 4 – Retrospektiv 2	37
8.5	Vedlegg 5 - Krav fra oblig 3.....	38
8.6	Vedlegg 6 - Krav gruppert etter domene	38
8.7	Vedlegg 7 - Google Maps Github Issue og Version 8.3.0 Release	41
8.8	Vedlegg 8 – Skjermbilder av appen.....	42

1 Presentasjon

1.1 Introduksjon/sammendrag

I denne rapporten beskrives team 20 sitt prosjektarbeid i IN2000 våren 2026.

Kjernen i prosjektarbeidet er en Android app som benytter data fra Meteorologisk institutt. Vi redegjør for case, applikasjonen, produktet, prosessen og våre refleksjoner rundt arbeidet.

1.2 Presentasjon av caset

Case 3: Naturhendelser har fokus på klima, definert som tilstanden et bestemt sted over lengre tid, og hvordan klima endres med global oppvarming. Casebeskrivelsen trekker frem klimatilpasninger med hensikt å spare penger, begrense skader ved naturhendelser som flom og ras. Beskrivelsen listet opp en rekke naturhendelser, der vi valgte å fokusere på flom fra vassdrag, styrtregn, kraftig vind og skred. Som målgruppen valgte vi boligkjøpere som primærbrukere, og boligeiere og boligbyggere, og eventuelt kommersielle aktører som sekundærbrukere.

1.3 Kort beskrivelse av løsningen

Løsningen er en app utviklet for å gi boligkjøpere en samlet vurdering av geologiske og meteorologiske forhold om ønsket lokasjon og lettere kunne ta dette med i betraktning ved kjøp av bolig. Kjernen i appen er det vi kaller GeoScore, en sammensatt score som beskriver hvor utsatt en gitt lokasjon er for naturhendelser som vind, nedbør, skred og flom. I tillegg viser appen historiske klimadata for valgt lokasjon visualisert med grafer.

1.4 Beskrivelse av teamet

Teamet består av seks studenter, der fem studerer Informatikk: Programmering og systemarkitektur (prosa) og én Informatikk: Design, bruk interaksjon.

Designstudenten har hatt rollen som UX designer og product owner. En av prosastudentene har også tatt IN1050 som er introduksjonsfaget til design. En annen prosastudent har stor interesse for frontend og har tatt hovedansvar for implementasjonen av UI som lead frontend developer. De resterende prosa-studentene har stor interesse for programmering, beregning og problemløsning.

Våre ulike personligheter, samt faglige bakgrunner, har gjort at vi utfyller hverandre både faglig og sosialt.

2 Brukerdokumentasjon

2.1 Applikasjonens funksjonalitet

Appen er bygget rundt en enkel og minimalistisk flyt med seks skjermer: hjem-, søke-, kart-, rapport-, historisk klimadata- og lagretskjerm.

Hjemskjermen er brukerens første møte med appen. Den inneholder en kort velkomsttekst som forklarer hva appen tilbyr, et søkefelt og en navigasjonsbar som gir tilgang til kart og lagrede steder.

Ved å trykke på søkefeltet navigeres brukeren til søkeskjermen. Her søkes det på adresser, ikke steder eller kommuner. Dette er et bevisst valg, siden appen er ment å gi en risikovurdering av en konkret adresse/lokasjon, ikke et større område. Når brukeren velger en adresse fra forslagslisten, blir de navigert til kartskjermen. Alternativt kan brukeren navigere direkte i kartet og trykke på en ønsket lokasjon. Dette behandles på samme måte som en søkt adresse og gir tilgang til den samme flyten videre.

Kartskjermen fungerer som et samlingspunkt i appen. Den gir en visuell bekreftelse på at riktig lokasjon er valgt med en pin, og brukeren kan herfra veksle mellom kartlag som viser flom-, skred- og steinsprangsoner for å få et overblikk over faresonene i området. Fra kartskjermen genererer brukeren en rapport ved å trykke på «Vis rapport».

Rapportskjermen inneholder hovedfunksjonaliteten i appen. Her presenteres GeoScoren og detaljert informasjon om de ulike risikofaktorene, med hver sin egen score, for det valgte stedet. For risikoer med score lavere enn B inkluderer rapporten konkrete tiltak for klimapasninger, generert av ChatGPT 4.0, sammen med en lenke til NVE for ytterligere informasjon fra en faglig kilde.

Fra rapportskjermen kan brukeren navigere til skjermen for historisk klimadata. Her vises grafer basert på målte klimadata fra perioden 1991 til 2020, som dekker gjennomsnittlig temperatur, nedbør, vind, snødybde og antall soltimer.

Direkte fra rapportskjermen kan brukeren lagre en lokasjon. Lagrede steder samles på lagretskjermen, hvor GeoScorene kan stilles opp mot hverandre for enkel sammenligning. Det er også mulig å fjerne lagrede steder.



2.2 Målgruppe

Primærbrukere: boligkjøpere

Sekundærbrukere: boligeiere og boligbyggere, eventuelt også kommersielle aktører

Målgruppen for denne appen er primært privatpersoner som vil ha innsikt i geologiske og meteorologiske forhold til en bolig de vurderer å kjøpe, og sekundært boligeiere og boligbyggere. Appen kan også være til nytte i det kommersielle virksomheter. Appen bruker per nå datakilder begrenset til Norge, men med muligheter for utvidelse ved eventuell videreutvikling.

2.3 Plattformen og aksessering

Applikasjonen kjører på Android-operativsystemet. Appen krever Android 7.0 (SDK 24) eller nyere, noe som gjør den kompatibel med 95,7% av aktive Android-enheter.

Et hinder for aksessering i Norge kontra globalt er at i Norge har iOS (Apples operativsystem) høy markedsandel (StatCounter Global Stats, u.å.). Ved videre utvikling, uten rammene som er satt av IN2000, ville vi migrert over til multiplattform. Det ville gjort at appen kunne bli tatt i bruk på både Android og iOS uten å måtte vedlikeholde og utvikle to disjunkte løsninger. En tilnærming for multiplattform som hadde blitt sterkt vurdert er kombinasjonen av Kotlin Multiplattform og Compose Multiplattform. Da er koden for både appens logikk og UI delt.

Per nå er ikke appen publisert. Den vil nå måtte distribueres til brukere via en sendt APK. For sensur vil applikasjonen aksesserer ved at zipfilen pakkes ut og gradle bygger appen.

3 Produktdokumentasjon

3.1 Kvalitetssegenskapene ved appen

3.1.1 Brukskvalitet

3.1.1.1 WCAG - Web Content Accessibility Guidelines

I utviklingen har vi hatt som mål å følge WCAG-kravene, sortert inn i fire prinsipper: mulig å oppfatte, mulig å betjene, forståelig og robust (Tilsynet for universell utforming av ikt, u.å.).

Med hensyn til prinsippet *mulig å oppfatte* har vi gjort flere bevisste valg under utviklingen av appen. Appen støtter at brukere skal kunne velge visningsretning, stående eller liggende (krav 1.3.4). Videre sørget vi for at kontrastfarger for tekst skal møte kravet på 4,5:1 (krav 1.4.3), og at det er tilbudt tekstlig beskrivelse av ikke-tekstlig innhold som ikoner (krav 1.1.1). Likevel har andre visuelle komponenter, slik som diagrammene og kart, ikke tekstlig tolkning da dette var utfordrende å implementere, grunnet store variasjoner av generert data.

For at appen skal være *mulig å betjene* har det vært viktig at appen støtter ulike måter å navigere på (krav 2.4). Blant annet har vi utviklet appen slik at fokusrekkefølgen er logisk og meningsfull for ulike orienteringer og skjermstørrelser (krav 2.4.3). Vi har også utviklet alternativ navigering, slik at brukere som navigerer sekvensielt ikke må iterere over den samme mengden handlingsknapper for å navigere lister (krav 2.4.1). Ved videre utvikling ville vi også sørget for at brukere kan hoppe over hele lister.

For at appen skal være *forståelig* og brukervennlig, har vi inkludert ledetekster og instruksjoner for en rekke brukeroppgaver (krav 3.3.2), eksempelvis søkefelter og navn på ikoner i navigasjonsbaren. Tekstlig informasjon er skrevet på et forståelig språk, slik at brukerne ikke må ha faglig kunnskap for å navigere seg gjennom appen.

I appen har vi hatt som mål å møte retningslinjene for *robusthet* knyttet til statusbeskjeder. Bruker får ventemelding for når data lastes, feilmelding for utilgjengelig data, og varsling som ber bruker bekrefte sletting av data lagret av bruker (krav 4.1.3).

3.1.1.2 *SavedState*

Et frustrasjonsmoment i appbruk er når appen mister konteksten fra brukerens økt. Det kan være flere utløsende årsaker. En kan være at data-holdere i appen ikke holdes i live gjennom økten. Det vil si at de ikke er skopet til å omslutte de komponentene som konsumerer dataen deres. Dette har vi unngått ved å presisere skopet. En annen årsak er system-initiert prosessdød. Om bruker går ut av appen, ved for eksempel anrop, kan enhetens system avslutte appen i bakgrunnen for å

frigjøre ressurser. Vi håndterer dette ved en navigasjonsløsning (Navigation 3) som gjenoppretter brukerens skjerm og en prosessdød-persistent tilstandsholder (Parcelable). Om en instans drept av systemet åpnes igjen via «Nylige-apper» (typisk det venstre ikonet i Android navigasjonsbaren) vil data fra før prosessdøden gjenopprettes.

3.1.2 Respons

Tidligere iterasjoner av appens algoritme for å generere GeoScore, det vil si den aggregerte og vektete vurderingen av forholdene, brukte ganske lang tid. Vi målte hastighet ved å logge kallet til GeoScore use-cases start og slutt. Vi fant at det langt overskred rammene funnet i empirien (Nielsen, u.å.).

Steg en var å parallelisere og effektivisere koden som genererer GeoScore. Dette gjorde utregningen kjappere og var en påminnelse om å få IO-arbeid av UI-tråden. Selv når algoritmens hastighet var optimalisert var ventetiden fremdeles litt lang. Så vel som den reelle lastetiden er også psykologien rundt lastingen viktig. Vi hadde en spinner, men denne kjentes litt paff. En løsning vi fant som fungerer ganske bra er en lastetekst (i tillegg til spinneren) som endrer seg gjennom ulike setninger. Den gjør dette helt vilkårlig, men lastetiden oppleves kortere.

3.1.3 Vedlikeholdbarhet og pålitelighet

Et gjennomgående mål under utviklingen av prosjektet har vært at kodebasen skal være mulig å videreutvikle og vedlikeholde uten uforholdsmessig stor innsats. Metoder og variabler er navngitt beskrivende etter funksjon, og arkitekturen følger MVVM-mønstre med tydelig separasjon mellom UI, domenelogikk og datalag. Avhengigheter håndteres gjennom Hilt, som eliminerer manuelt dependency-håndtering og sikrer typsikkerhet ved kompilering (Android Developers, u.å.).

Gjennom code smells-aktiviteten og refaktoreringen ble metoder med for mange ansvarsområder delt opp i hjelpemetoder, og død kode samt akkumulerte logger ble fjernet i slutfasen.

Appen er bygget med lav kobling som mål, der alle repositories er definert som grensesnitt i domenelaget slik at øvre lag forholder seg til abstraksjonene og ikke til konkrete implementasjoner. Det finnes likevel unntak der høyere kobling ble et bevisst valg. GeoScore-beregningen kaller på flere use cases parallelt, noe som øker koblingen, men som var nødvendig for å holde responstiden innenfor akseptable rammer.

For pålitelighet er feilhåndtering implementert ved alle nettverksavhengige operasjoner. Dersom et API-kall feiler eller nettverket er utilgjengelig, varsles brukeren og gis mulighet til å prøve på nytt. Enhetstestene dekker ViewModels, repositories og datakilder, og integrasjonstester gjør reelle API-kall for å verifisere sentrale use cases ende til ende.

3.2 Brukerundersøkelser og metoder for datainnsamling og analyse

For datainnsamling benyttet vi oss av intervju som metode. Vi intervjuet totalt fire brukere. To av dem hadde erfaring med kjøp av bolig, to var på boligjakt. Intervjuene ble en semistrukturert variant med stikkord og spørsmål, men der noen ble hoppet over og andre nye spørsmål oppstod. Som supplement brukte vi lavoppløselige prototyper som kommunikasjonsverktøy. For å nevne noen funn: skalaen ga ikke mening å være tall 1-100 som flere av prototypene brukte, men heller være A-G lignende det folk flest kjenner fra før av med energimerking, eller «Lav – Moderat – Høy» som beskriver mer tolkningen av tallene. En av brukerne var dog mer opptatt av detaljene og bakgrunnen for merkingen, mer enn tallet, bokstavene eller merke et sted har, og vektla derfor ønsket om å ha en god forklarende utdypning i rapporten for hver av faktorene merkingen baserer seg på. Brukerne ønsket HomeScreen/førstesiden å være ren og med lite innhold, kun et søkefelt som det første de ville møte i appen. En bruker gjorde oss også oppmerksom på at merkingen ikke burde endre seg basert på hvilket tidsperspektiv man velger, men at dette er en statisk verdi som ikke endrer seg, kun ved eventuelt avdekking av ny data på permanente forhold, som for eksempel ny kartlegging av berggrunn. En tredje bruker hadde mer domenekunnskap innenfor geologi og arealplanlegging, og tipset oss om andre kilder til data vi kunne ha nytte av, blant annet PlacePoint. Valg av navn på appen fikk vi også tilbakemelding på, der flere likte «Geomerking», mens

andre ønsket noe mer som «Fare-» eller «Risikomerking». Det var enstemmig enighet om å ha en side med lagrede steder. Generelt var brukerne positive til ideen, likte og så nytteverdien av konseptet.

3.3 Brukte API

Appen henter data fra Meteorologisk institutt og flere eksterne tjenester, inkludert Frost API v0 og v1 (Meteorologisk institutt), Geonorge sitt adressesøk, NVE sin karttjeneste, OpenAI sitt ChatGPT API og ArcGIS sine API-er for flom og skred.

3.3.1 Meteorologisk Institutt

Frost API er den sentrale datakilden for klimainformasjon i appen. Vi bruker begge versjonene av API-et, men til ulike formål. Frost v0 brukes for historisk klimadata, ettersom aggregerte data som månedlig middeltemperatur og nedbør per døgn ikke er tilgjengelig i v1 ennå. Frost v1 brukes for observasjonsdata som inngår i beregningen av GeoScore. Fra Frost henter vi klimanormaler for perioden 1991 til 2020, herunder temperatur, nedbør, vindstyrke, snødybde og solskinnstimer.

Gjennom bruk av Frost møtte vi på flere utfordringer. Den største og den som har vært mest krevende å håndtere er datagap i værstasjonene. Ideelt sett ville alle stasjoner ha målt all data i alle perioder, men slik er det ikke. For å gjøre appen mindre sårbar for dette og for å redusere antall API-kall, aggregerer vi data fra de 10 nærmeste værstasjonene for en lokasjon. Dette gir oss robuste og mer generelle klimanormaler. Klimadataene caches så lokalt i Room-databasen med stasjons-ID som nøkkel, slik at gjentatte søk i samme område ikke genererer nye API-kall.

Solskinnsdata er et spesialtilfelle, det er kun 36 stasjoner i Norge som målte dette i det aktuelle tidsrommet. Nærmeste solskinnstasjon slås opp separat og caches på samme måte som øvrige klimadata.

3.3.2 Andre API-er

Geonorge sitt adressesøk benyttes for å oversette adresser til koordinater. Tjenesten returnerer strukturerte adresseforslag basert på brukerinput i søkefelt, og

koordinatene til det valgte stedet returneres og brukes som grunnlag for alle videre API-kall.

NVE sine karttjenester brukes for å vise kartlag av flom og skredsoner i kartskjermen. Vi bruker i tillegg NVE sin ArcGIS-baserte API på å undersøke om en lokasjon ligger i en flom eller skredsone, dette inngår i grunnlaget for GeoScore beregningen.

ChatGPT 4.0 API fra OpenAI brukes til å formulere den tekstlige rapporten som presenteres for brukeren i rapportskjermen. Rapporten genereres på bakgrunn av klimastatistikk og GeoScore for den valgte adressen, og gir brukeren en lesbar oppsummering av risikoforholdene, samt forslag til tiltak basert på hva stedet scorer dårlig på. Genererte rapporter caches lokalt for å unngå unødvendige API-kall ved gjentatte besøk på samme adresse.

3.4 GeoScore-algoritmen

3.4.1 Beskrivelse av algoritmen

Algoritmen evaluerer risikoen knyttet til en gitt geografisk lokasjon, representert ved koordinater, og produserer en GeoScore basert på tre uavhengige delscorer beregnet på 30 års historiske klimadata. Hver delscore er normalisert til en skala fra 0–100, der høyere verdier indikerer høyere risiko.

Delscorene dekker til sammen et bredt spekter av risikofaktorer, inkludert naturhendelser som styrtregn, flom, kraftig vind og skred.

De tre delscorene aggregeres til en samlet GeoScore gjennom en vektet sum. Vektfordelingen mellom delscorene er en faglig vurdering som vi definerte selv, men som ideelt sett burde ha vært en beslutning fastsatt av en kvalifisert fagperson, for å sikre at den samlede scoren reflekterer reell meteorologisk og geografisk risiko på en forsvarlig måte.

3.4.2 Del-scorer

GeoScore-algoritmen er bygget opp av tre delscorer, hver av disse måler en distinkt del av risiko ved en gitt lokasjon.

HazardScore – Intensitet

HazardScore måler intensiteten av en naturhendelse på tidspunktet den inntreffer. En naturhendelse defineres som et tidspunkt der vind og/eller nedbør overskrider fastsatte terskelverdier, hentet fra Meteorologisk institutt (Uthus, 2021).

VulnerabilityScore – Geografisk Sårbarhet

VulnerabilityScore angir om en lokasjon ligger innenfor kartlagte skred- eller flomsoner. Scoren reflekterer dermed den geografiske eksponeringen for kjente fareområder, uavhengig av hvor ofte eller hvor kraftige hendelser som forekommer.

ExposureScore – Eksponering Over Tid

ExposureScore kvantifiserer hvor mange naturhendelser en lokasjon er utsatt for over tid, relativt til en egendefinert maksimumsverdi (MAX_EXTREME_WEATHER_COUNT). I gjeldende algoritme er denne satt til 15 naturhendelser per år, et foreløpig estimat som ideelt bør fastsettes av en meteorolog. Naturhendelser telles etter samme definisjon og terskelverdier som benyttes i HazardScore, hentet fra Uthus (2021).

3.4.3 Begrunnelse for valg av naturhendelser

Algoritmens fokus på styrtregn, flom, kraftig vind og skred er bygget på dokumentert skadestatistikk. Ifølge Finans Norges klimarapport 2025 står disse naturhendelsene samlet for over 75% av alle forsikringsskader i Norge (Finans Norge, 2025), noe som støtter deres sentrale rolle i en risikomodell rettet mot geografisk og meteorologisk sårbarhet.

Ved å konsentrere algoritmen om de hendelsestypene med størst dokumentert skadepotensial sikres både relevans og treffsikkerhet i risikovurderingen. Beregningene tar utgangspunkt i de siste 30 årene, et tidsspenn som gir et representativt bilde av klimatiske mønstre uten at datagrunnlaget blir uforholdsmessig stort.

3.5 Databasen

Applikasjonen bruker Room som lokal database for å lagre informasjon om steder brukeren har valgt å inspisere. Når en bruker velger en lokasjon og henter ut stedsdata, lagres resultatet lokalt slik at gjentatte oppslag mot samme område slipper å trigge nye beregninger og API-kall.

Datalagringen dekker tre kategorier, scores, historisk klimadata og lagrede lokasjoner. For scorene hentes rådata fra Frost V1 APlet (vind og nedbør) og ArcGIS-APIene (flom og skredsoner). Disse aggregeres til en nedbørsscore, vindscore, flom- og skredscore, som igjen danner grunnlaget for Hazard-, Exposure- og Vulnerability-scorene samt en samlet GeoScore. Aggregeringslogikken er beskrevet nærmere i algoritmekapittelet. Alle scorene lagres på lokasjonens locationkey, som er de abstraherte koordinatene. Mer om det nedenfor.

For historisk klimadata hentes informasjon fra de ti nærmeste meteorologiske stasjonene. Dataen lagres på den nærmeste stasjonen, noe som i noen tilfeller kan medføre duplikatdata siden flere stasjoner kan rapportere overlappende målinger. Per værtype, som snø, vind og soltimer, beregnes tolv aggregerte datapunkter, en til hver måned, som brukes til å fylle linjediagrammene i applikasjonen og lagres også lokalt.

Den tredje kategorien er lagrede lokasjoner. Brukeren kan aktivt lagre steder, og da lagres adresse, koordinater, sted, kommune og tidspunkt lokasjonen ble lagret lokalt på enheten. Disse henter ikke inn scores eller klimadata på nytt, de benytter seg av den samme "locationkey" (abstraherte koordinater) strukturen for å hente ut eksisterende data om lokasjonen som allerede er beregnet.

Koblingen mellom lagret data og geografisk lokasjon løses ved å abstrahere koordinatpresisjonene. En adresse som brukeren velger, har opprinnelig koordinater med fem desimaler. Ved lagring rundes disse ned til to desimaler, som tilsvarer en radius på omtrent én kilometer. Begrunnelsen er at lokasjoner innenfor denne avstanden vil gi tilnærmet identiske scores og klimadata, og det gir liten gevinst å beregne på nytt. Valget innebærer et bevisst kompromiss mellom presisjon og

brukervennlighet, der brukervennlighet ble prioritert fordi beregningene er ressurskrevende og tar merkbart tid.

3.6 Kart

Applikasjonen viser data fra NVE i et kart for å vise diverse naturfarer grafisk. Som karttjener valgte vi Google Maps Compose siden det fungerer sømløst med Compose samt at det var gratis. Et problem med dette var at Google Maps Compose er laget for tiled web maps, men NVE sine data serveres på web map service. De grunnleggende dataene er på et kompatibelt format, men krever en oversettelse av tiled web map spørringer til web map service spørringer. Dette løste vi ved å skrive en algoritme for å oversette x/y/z spørringer til bounding box (BBOX) spørringer.

For fleksibilitet lagde vi en dedikert wms-overlay composable som tok en lambda, en funksjon, som ga den oppskriften på hvordan URL-en skal settes sammen med de beregnede BBOX-koordinatene.

En grunn til at denne konverteringen ikke nødvendigvis tilbys som standard av biblioteker er at det er forskjeller i hvordan ulike web map services krever forespørselen, så en rigid konvertering kan lett lage ugyldige URL-er. Vi innså at vår løsning hadde vært passende å ha i standard biblioteket så vi åpnet en issue på Google Maps Compose sin GitHub. Bidragsyttere i dette biblioteket (en frivillig og en google ansatt) implementerte så sin egen kode etter formen av vår løsning. Koden ble merget inn i main og publisert i versjon 8.3.0 av Google Maps Compose. Vi tok så denne i bruk og slettet vår egen konverterer fra appen ([vedlegg 5](#)).

Historisk klimadata – Compose Charts

For visualisering av historisk klimadata bruker appen biblioteket Compose Charts av Ehsan Narmani (Narmani, 2024). Biblioteket tilbyr linjediagrammer som er godt tilpasset Jetpack Compose og støtter flere linjer i samme diagram, noe som gjør det mulig å vise for eksempel minimums-, gjennomsnitts- og maksverdier samtidig. Diagrammene er konfigurert med interaktive datapunkter og tilpassede farger for å gjøre det enkelt å skille de ulike verdiene fra hverandre.

4 Prosessdokumentasjon

4.1 Utviklingsmetode og bruk av smidige praksiser

Teamet benyttet Scrum som utviklingsmetode gjennom hele prosjektet, organisert i ukentlige sprinter. Notion fungerte som felles informasjonsplattform og inneholdt en Kanban-tavle for product backlog, mens GitHub ble brukt til versjonskontroll og kodesamarbeid. Selve utviklingen foregikk i Android Studio, og design ble utarbeidet i Figma. For datavisualisering og diagramarbeid ble henholdsvis draw.io og Mermaid benyttet. Akademiske kilder ble håndtert gjennom Zotero, og Word ble brukt til rapportskriving. Claude og GPT UiO ble benyttet som KI-støtteverktøy underveis.

For kommunikasjon ble rollene delt mellom to plattformer: Messenger håndterte den løpende, uformelle dialogen mellom teammedlemmene, mens Microsoft Teams ble brukt til møteplanlegging og kommunikasjon med veiledere.

Rollene ble fordelt slik at product owner-ansvaret lå hos ett teammedlem gjennom hele prosjektet. Product owner hadde også i første halvdel av prosjektet rollen som scrum master, men etter påske rullerte vi denne ukentlig fordi flere i teamet ønsket erfaring med denne rollen. Den faste møtestrukturen bestod av et firetimers mandagsmøte, og etter hvert også et fast to til fire timers fredagsmøte med stand-up. Utviklingsmetoden ble diskutert kontinuerlig, og særlig tatt opp på de to retrospektivene vi hadde jevnt fordelt i perioden.

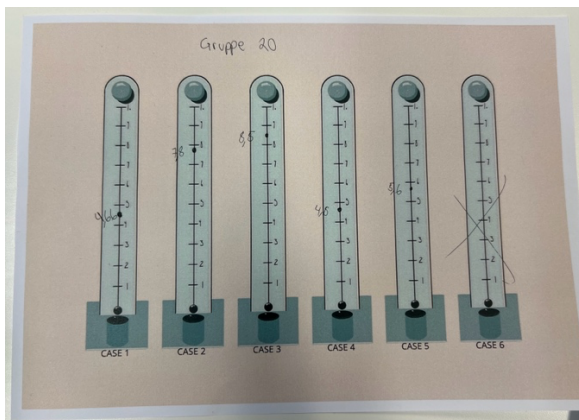
Tidlig i prosjektet manglet sprintene koordinerte mål på tvers av roller, noe som førte til ujevn fremdrift. Dette ble løst ved å samkjøre sprintmål mellom designere og frontend- og backendutviklere, noe som resulterte i mer helhetlige leveranser per sprint.

4.2 Prosjektets gang

4.2.1 Kick-off og valg av case

Prosjektet startet med kick-off der vi ble presentert for årets caser, møtte teamet for første gang, og ble kjent gjennom blant annet «stinky fish» og pizza. Vi idémyldret rundt casene og det kom fort frem at en av teammedlemmene hadde en app-idé med geomerking fra tidligere som passet svært godt til case 3. Idemyldringen

avsluttet med en individuell rangering av casene før de ble samlet til et felles case-barometer.



Basert på idemyldringen og case-barometeret, landet valget fort på case 3.

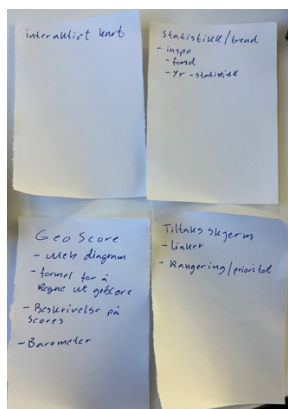
4.2.2 Oppstartsfasen/Sprint 0

Oppstartsfasen ble brukt til å etablere det sosiale og tekniske grunnlaget for prosjektet. For å bli bedre kjent med hverandre dro hele teamet på minigolf på Oslo camping allerede mandag etter kick-off. I tillegg ble det utarbeidet en teamavtale som definerte forventninger og retningslinjer for samarbeidet ([Vedlegg 2](#)). På den tekniske siden ble Android Studio prosjektet og GitHub repository satt opp, og klargjort for utvikling. En konfigurasjonsfeil i det initielle prosjektoppsettet skapte tekniske problemer tidlig i utviklingsfasen. Ideen om geomerking utforsket vi videre i en workshop med «Crazy 8». Her fikk vi en rekke ideer til utformingen av appens skjermer og funksjonene de kunne ha.



4.2.3 Iterasjon 1

I første iterasjon var fokus på å definere og spisse oss inn på målgruppe. Med resultatene fra «Crazy 8» var vi innom både turgåere og aktører i skog- og landbruket, men landet på boligbyggere og boligeiere. Grunnet endring av casebeskrivelsen med mindre fokus på varslinger og mer på historisk klimadata, byttet vi målgruppe og valgte boligkjøpere som primærbrukere. Øvelsen med «crazy 8» ga oss også et bredt utvalg av ideer vi tok med oss inn i kravspesifikasjonene. Blant funksjonene vi tok med oss videre var et interaktivt kart, statistikk for å visualisere klimatrender, en rapport med GeoScore og en tiltaksskjerm.



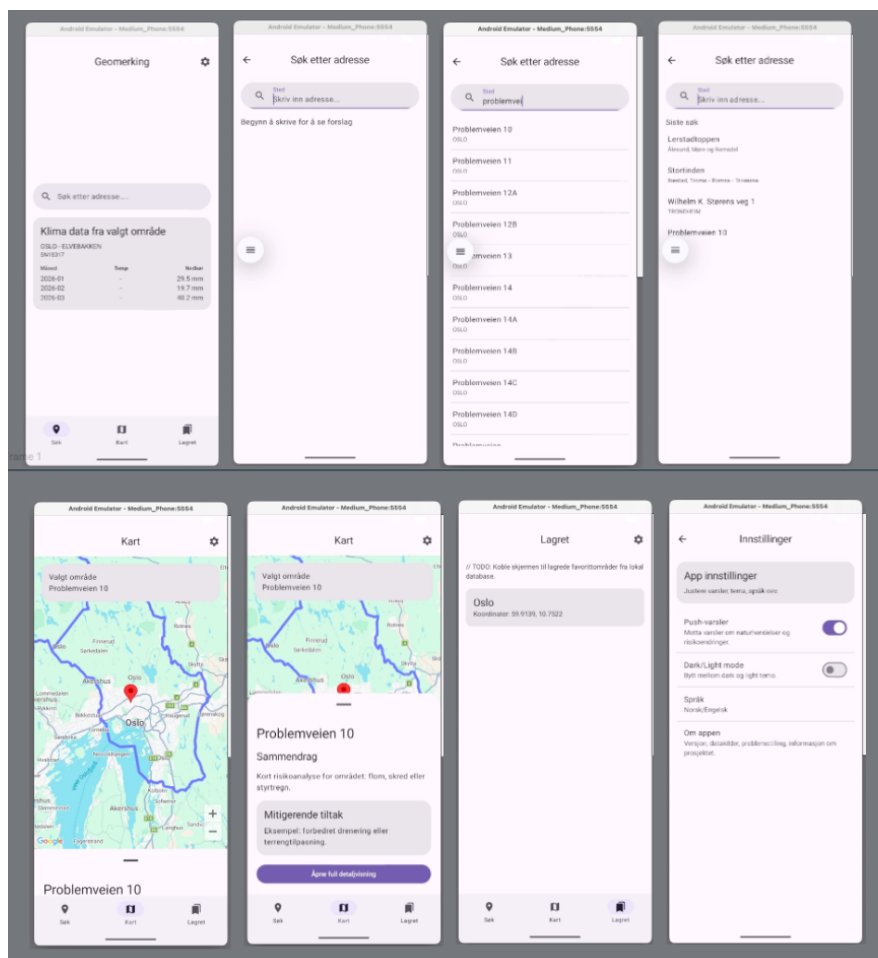
Vi begynte å sette opp skjelettet for de ulike skjermene, samt så på ulike API-løsninger og implementering av Frost.

På designsidene begynte vi med lavoppløselige prototyper i form av enkle skisser på papir og nettbrett, supplert med vibe-kodede interaktive prototyper. Disse ble brukt som nyttig kommunikasjonsverktøy i brukerintervju med totalt fire brukere. Innsikt fra brukerintervjuet brukte vi til å lande på tre hovedskjermer, der første skulle være en

minimalistisk og ren skjerm med søkefeltet i fokus. Vi gikk bort ifra funksjonen å kunne endre valgt tidsperspektiv da GeoScoren skal speile forhold som ikke endrer seg, tillegg til at skala ikke skulle være fra 1-100, men en skala som i større grad tolker dataen, slik som «lav, moderat, høy» eller A-F. Vi bestemte oss for å ha en lagret-skjerm for rapporter, og at rapportene skulle inneholde detaljert informasjon om bakgrunnen for GeoScoren. Appens navn ble også diskutert, og forslag som risikomerking og faremerking kom inn som forslag uten at vi spikret navnet på dette stadiet.

4.2.4 Iterasjon 2

I andre iterasjon nådde appen et Minimum Viable Product (MVP) med implementerte skjermene basert på funn fra brukerintervjuene, samt Frost v0 med visuell visning av dataen.



Vi gjennomførte vår første retrospektiv der vi avdekket flere forbedringspunkter for videre arbeid ([Vedlegg 3](#)). Vi ønsket oss tydeligere spesifiseringer av krav, tydeligere

sprinter, samt bedre struktur på møtene våre. Dette resulterte i konseptet «Lappen» der alle på starten av møte fikk tildelt en lapp til å skrive ned punkter man kom på underveis og ønsket å ta opp. I hver møteplan ble det satt av tid til å ta frem lappen. I tillegg åpent vi opp for å komme med innspill til møteplanen og sørget for enighet rundt den før møtene startet. Dette forebygget avsporinger og distraksjoner fra temaene vi diskuterte. Selv om vi hadde bestemt oss for ukentlig sprinter, hadde vi fortsatt behov for tydeligere avklaring på hva hver sprint inneholdt og når en ny begynte. Heretter hadde vi fast sprint review, sprint planning og sprint goal, for hver sprint, og dokumenterte dette i Notion.

Teamaktiviteten med backlog refinement viste seg å være akkurat det vi trengte på tidspunktet vi gjennomførte den. Her satte vi opp en oversikt over alle krav, både for caset, dokumentasjon, produktet og vår casespesifisering. Vi benyttet oss av brukerhistorier, personas og scenario for å tilrettelegge for bedre felles forståelse av appen. Prototyping i Figma kom her også godt i gang og ga oss en visuell fremstilling av appen slik vi så den for oss på dette stadiet, og bidro til alignment i teamet. Vi la inn interaksjoner slik at prototypen kunne brukes til geriljatesting før vi implementerte flyten i appen.

Parallelt begynte teamet å skissere algoritmen. Fremfor å dekke flest mulig naturhendelser avgrenset vi utvalget til hendelsene som forårsaker størst skade og som det finnes mest tilgjengelig data på. Skissen ble presentert for veileder før implementering, og tilbakemeldingene var gode nok til at vi gikk videre med den uendret.

4.2.5 Iterasjon 3

Implementeringen av universell utforming (UU) ble gjennomført tidlig i iterasjon 3, men tidspunktet viste seg å være uheldig. Siden implementeringen ble påbegynt sent i prosjektet, krevde det omfattende refaktorering av eksisterende kode. Den akkumulerte tekniske gjelden førte til mange merge-konflikter og en kontinuerlig refaktorering prosess videre i prosjektet. Den tekniske gjelden som kom med implementasjon av UU brukte mer tid enn ønsket, som førte til at vi måtte prioritere ned tid til andre oppgaver.

Den andre retrospektiven kom tett på og oppstod av behov. Etter en periode med mye stress og noen konflikter kjente teamet på et behov for å rydde opp.

Retrospektiven ga rom for å avklare misforståelser og konflikter direkte. Resultatet var en felles oppklaring som stabiliserte samarbeidsdynamikken i den avsluttende fasen av prosjektet ([Vedlegg 4](#)).

Frem til iterasjon 3 benyttet vi ViewModelFactories for å håndtere Dependency injection. Etter hvert som prosjektet vokste, ble det klart at denne tilnærmingen ikke skalerte godt. Dependency-grafen ble større og vanskeligere å vedlikeholde. Teamet valgte derfor å gjennomføre en større refaktorering der ViewModelFactories ble erstattet med Hilt, et dependency injection rammeverk som automatisk håndterer opprettelse og livssyklus for avhengigheter på tvers av appen (Android Developers, u.å.).

Brukertestene ble gjennomført mot slutten av iterasjonen, på et produkt som var nærme slutfasen. Vi gjennomførte brukertest med totalt tre brukere der alle var innenfor vår målgruppe. Testene var nyttige for å teste forståelse av appen, dens intuitivitet og brukeropplevelse, tillegg til flere mindre bugs som var fint å avdekke. Funnene resulterte også i en rekke endringer i flow og funksjoner. For eksempel behovet for navigasjonsbar på alle skjermer, og en tool tip med informasjon om skalaen brukt i rapportene. En av brukerne ønsket muligheten til å sammenligne perioden 1991–2020 med en tilsvarende tidligere periode, for å bedre forstå klimaendringene.

4.2.6 Slutfasen

I slutfasen ble funnene fra brukertestene gjennomgått og prioritert. Vi valgte å prioritere navigasjonsbaren, mens filtrering og utvidelse av klimaperioden ble nedprioritert grunnet tidsrammene.

I denne perioden tilbragte teamet svært mye tid sammen. Vi hadde hyppigere og lengere møter. Vi erfarte at vi var langt mer effektive når vi jobbet i samme rom enn hver for oss. På møtene etablerte vi en fast rytme der vi jobbet i 50-minutters økter med 10 minutters pause. Hver på teamet hadde sin egen lapp der spørsmål og uklare punkter ble notert ned underveis. Disse ble så tatt opp i 10 minutters pausene

med standup, beinstrekk og gjennom av lappene. Dette bidro til svært effektivt arbeid med hyppig alignment og færre avbrytelser i arbeidet.

Arkitekturskissen fra MVP måtte oppdateres for å reflektere tilstanden til kodebasen. Vi utarbeidet i tillegg to use-case-diagrammer, et sekvensdiagram, et aktivitetsdiagram og et klassediagram. Arbeidet var lærerikt, og diskusjonene rundt diagrammene ga oss en bedre forståelse av arkitekturen. Avslutningsvis ryddet vi opp i akkumulert død kode og logging. I denne prosessen ble vi oppmerksomme på at en ny GeoScoreViewModel ble initialisert for hver lokasjon, og at vi hadde glemt det funksjonelle kravet:

«Applikasjonen skal inkludere en tydelig ansvarsfraskrivelse om at GeoScoren er et estimat og ikke erstatter profesjonell rådgiving.»

Begge ble rettet på innleveringsdagen.

4.3 Endringer i kravspesifikasjoner – hva og hvorfor

De funksjonelle og ikke-funksjonelle kravene som ble satt i den tidlige fasen av prosjektet tok utgangspunkt den originale casebeskrivelsen, men reflekterte på det tidspunktet en bredere og mer værorientert forståelse av oppgaven enn det som etter hvert viste seg å være riktig retning ([Vedlegg 5](#)).

Gjennom de første ukene ble det tydelig at flere av disse kravene pekte mot kortsiktig værvarsling fremfor klimainformasjon over tid. En presisering av casebeskrivelsen poengterte at appen ikke skulle være et verktøy for å sjekke været, men heller formidle historiske klimadata og bidra til forståelse av klimatilpasning. Dette var en viktig ting som førte til at kravene var nødt til å revideres.

Endringen var ikke bare teknisk, men også konseptuell. I den tidlige fasen hadde teamet ideer som gikk i retning av sanntidsvarsling, risikoskåring og akutt farevarsling. Gjennom iterasjonene ble det klarere at appen skulle bygge på to ting: klimastatistikk for et valgt område, og en stedsspesifikk GeoScore som oppsummerer klimarisikoen for en eiendom basert på historiske data.

De endelige kravendringene ble oppdatert etter backlog-refinement, der kravene reflekterte de nye beslutningene for appen.

4.4 Testing

Underveis i prosjektet har vi benyttet oss av ulike teststrategier både for testing av det tekniske og brukernes opplevelse av appen. For det tekniske har vi gjort fysisk testing, enhetstester og testing som utviklingsverktøy. For brukeropplevelse har vi brukt geriljatesting og brukertest av appen. Med en slik multistrategisk tilnærming har vi forsøkt å oppnå en app som fungerer både teknisk og med god brukeropplevelse.

4.4.1 Testing på enhet/unit testing

Vi har gjort en utbredt fysisk testing på diverse enheter og skjermstørrelser. Dette var et viktig verktøy i å gjøre at appen fungerte på de mange ulike enhetene som finnes. Særlig i utviklingen av adaptiv layout var tilgang på et nettbrett gunstig.

Vi har også implementert enhetstester for å verifisere at appen fungerer slik det er tiltenkt. Enkelte av våre enhetstester tester mot eksterne API-er. Dette har vi innsett er en smule uheldig siden testing av intern logikk ikke burde avhenge av eksterne kilder vi ikke har kontroll over.

Testing som utviklingsverktøy har og blitt prøvet ut i form av en TDD-aktivitet, Test Driven Development, hvor man skriver tester for forventet oppførsel *før* man skriver koden testene er for.

4.4.2 Brukertester

Geriljatesting krevde lite planlegging, men krevde derimot en testbar prototype, der vi brukte Figma med interaktive elementer slik at brukerne kunne navigere direkte som om det var en app. Vi gikk rundt på både IFI (Institutt for informatikk) og Fredrikke spiseri på UiO (Universitetet i Oslo), og Pust café på Majorstuen, der spurte vi de vi møtte på om de hadde et par minutter. Vi tok skjermopptak med lyd etter muntlig samtykke. Med oss hadde vi en testguide med konkrete skjermer og funksjoner vi ønsket tilbakemelding på. Til slutt kategoriserte vi brukersitatene, samlet funnene og lagde action points ut av dem ([Vedlegg 9](#)). Hensikten her var å test hvor intuitivt designet var, mer enn å avdekke nye funksjonelle krav.

Brukertestene ble nøyere planlagt med en mer detaljert testguide ([Vedlegg 10](#)) og avtalt tid og sted med brukerne. Det ble tatt skjermpptak med lyd etter skriftlig samtykke på samtykkeskjema ([Vedlegg 1](#)). Disse tok 30-60 minutter. Det var et fokus på funksjonelle krav, brukervennlighet og UI på fysisk enhet.

5 Refleksjon

5.1 Jobbe som tverrfaglig team

Gruppens sammensetning var prosa-dominant så det var veldig viktig for oss å jobbe tverrfaglig for å kunne utvikle god UI effektivt. Vi hadde da stor nytte av å kunne bruke designutviklers kompetanse til å vurdere teamets arbeid. I startfasen var skillet mellom rollene frontend(utvikling) og UI/UX(design/brukeropplevelse) mindre, men etter hvert som brukertesting ble mer integrert i designprosessen og det ble mer frontend arbeid ble rollene mer delt.

Brukertesting er derimot ikke rent design spesifikk så en utvikler deltok noe for å få input fra bruker angående ting som handlet mer om appens funksjon.

5.2 Smidig prosess og effekten av det

Scrum som metode viste seg å være effektiv, men effektiviteten kom ikke av seg selv, den måtte læres frem. Tidlig i prosjektet ble oppgaver definert og delegert uten et tydelig felles mål, noe som resulterte i at arbeid ble gjort litt her og der uten at delene hang naturlig sammen ved sprintens slutt. Etter hvert gikk teamet over til å formulere ett konkret sprintmål per sprint og samle all design-, UI-, backend- og planleggings- oppgaver rundt det målet. Dette skiftet merket vi tydelig på samkjøringen og resultatet av sprintene våre.

En av de viktigste innsiktene vi satt igjen med er at smidig utvikling forutsetter god kommunikasjon. Vi hadde lett for å utvikle ulike oppfatninger av produktet i starten av prosessen. Løsningen var å stoppe opp og diskutere problemer og avklare misforståelser i plenum. Dette førte til en kodeløstopp i prosessen som reflekteres i GitHuben vår. Etter at misforståelsene var avklart, og alle hadde en felles forståelse av produktet, gikk vi over til en smidigere smidig utviklingsprosess.

5.3 Sentrale utfordringer i utviklingsprosessen

En vedvarende utfordring var balansen mellom planlegging og programmering. Prosessen var preget av gjentatte stopp for avklaring underveis, noe som kunne vært redusert med en mer grundig oppstartsfase. Hadde vi gjort prosjektet på nytt, ville vi hatt mer fokus på felles forståelse kontinuerlig gjennom hele prosessen av hva som skulle bygges, hvordan det skulle se ut og hvordan det skulle fungere. Spesielt i en smidig prosess der krav endrer seg underveis.

5.3.1 Utfordringer med Github

Vi benyttet oss av trunkbasert utvikling som arbeidsmetode. Dette innebærer at all utvikling skjer i brancher som jevnlig flettes inn i mainbranchen. Målet er å unngå store og langvarige avvik mellom koden i main og pågående arbeid (Trunk Based Development, u.å.).

Dette fungerte stort sett bra, men vi var ikke alltid konsekvente nok med å få endringer inn i main etter at arbeidet var ferdig. I et tilfelle resulterte dette i at en branch med logikken for beregning av GeoScore algoritmen lå 38 commits bak main. Forskjellen mellom branchene var såpass stor at det ikke var et reelt alternativ med rebase. Vi valgte å flytte endringene manuelt over med støtte fra KI-verktøy. Dette var tidkrevende og understreket viktigheten av å holde levetiden på brancher korte og å merge inn i main ofte.

5.3.2 Tester

Testingen var et område der vi ikke klarte å opprettholde den praksisen vi startet med. Etter TDD-aktiviteten tidlig i prosjektet ble testene gradvis nedprioritert, og på et tidspunkt ble flere av testene kommentert ut og forsvant ut av fokus. Da filene ble gjenoppdaget sent i prosjektet viste det seg at de var utdaterte, ikke matchet gjeldende funksjonssignaturer og ikke testet den daværende funksjonaliteten i appen. I mellomtiden ble mye av feilsøkingen gjort manuelt ved hjelp av logging. Dette var tidkrevende, og vi ser i ettertid at oppdaterte tester ville ha avdekket flere

feil tidligere og gitt oss en bedre forståelse av koden underveis. Dette er en erfaring vi tar med oss videre.

5.3.3 Teknisk gjeld

Teknisk gjeld var en vedvarende utfordring som ble mer merkbar mot slutten av prosjektet. Krav om universell utforming og responsivt design ble ikke prioritert fra start, noe som gjorde dem mer krevende å implementere da de ble tatt tak i. Dette resulterte i flere merge-konflikter og refaktoreringsøkter som tok opp verdifull tid nærmere innleveringsfristen.

Tidlig i prosessen håndterte vi datamangel i Frost-stasjonene ved å hente data fra de 30 nærmeste stasjonene og ta gjennomsnittet. Dette ble senere nedjustert til 10 for å begrense antall API-kall og gjøre dataen mer presis. Ideelt sett burde vi for hver værtype ha valgt nærmeste stasjon som faktisk har den aktuelle dataen, slik vi gjør for solskinn. Det kan derimot hende at ingen av de 10 nærmeste stasjonene har f. eks. snødataen som vi etterspør, noe som gir tomme verdier fremfor et godt estimat.

API-nøkler er hardkodet i appen og lastet opp til det eksterne GitHub-repositoryet etter eksplisitt beskjed fra veileder om at dette var forventet leveranseformat. Dette er ikke praksis vi ville fulgt utenfor denne konteksten.

6 Avslutning

6.1 Det viktigste vi tar med oss videre

«Fått økt forståelse av viktigheten med alignment i prosjektarbeid. Når man har en klar oversikt over produktive ting å gjøre blir det mye lettere å jobbe.»

- David

«Det viktigste jeg har lært er hvor viktig kommunikasjon er i et gruppeprosjekt, og at man skal tørre å stille spørsmål fordi andre lurte garantert på det samme.»

- Jurius

«Det viktigste jeg kommer til å ta med meg fra prosjektarbeidet er smidige metoder, og bruk av API.»

- Matilda

«Jeg har lært mye om teamarbeid og hvordan jobbe med et større prosjekt over lengre tid.»

- Victoria

«Jeg har tilegnet meg ferdigheter ved bruk av Figma som designverktøy. Jeg tar med meg viktigheten av god møtekultur, tydelig rollefordeling og verdien av å jobbe i sprinter med sprint planning, sprint review og klare sprintmål.»

- Veronica

«Jeg har blitt mer oppmerksom på endringsfleksibiliteten man må ha rundt tentative planer.»

- Philip

6.2 Tips til neste års studenter

Vi vil anbefale github issues som kode koordinering. Beregn tid til å skrive om teamaktivitet i prosessloggen rett etter at aktiviteten er gjennomført.

7 Referanseliste

Android Developers. (u.å.). *Dependency injection with Hilt*. Android Developers.

Hentet 15. mai 2026, fra <https://developer.android.com/training/dependency-injection/hilt-android>

Finans Norge. (2025). *Klimarapport Finans Norge 2025*. Finans Norge.

<https://www.finansnorge.no/siteassets/dokumenter/publikasjoner/finans-norge-klimarapport-2025.pdf>

Narmani, E. (2024). *Overview—Compose Charts*.

<https://ehsannarmani.github.io/ComposeCharts/>

Nielsen, J. (u.å.). *Response Time Limits: Article by Jakob Nielsen*. Nielsen Norman

Group. Hentet 15. mai 2026, fra <https://www.nngroup.com/articles/response-times-3-important-limits/>

StatCounter Global Stats. (u.å.). *Mobile Operating System Market Share Norway*.

StatCounter Global Stats. Hentet 15. mai 2026, fra <https://gs.statcounter.com/os-market-share/mobile/norway/>

Tilsynet for universell utforming av ikt. (u.å.). *WCAG sortert etter prinsipper*. utilsynet.

Hentet 15. mai 2026, fra <https://www.uutilsynet.no/wcag-standard/wcag-sortert-etter-prinsipper/713>

Trunk Based Development. (u.å.). *Trunk Based Development*.

trunkbaseddevelopment.com. Hentet 15. mai 2026, fra <https://trunkbaseddevelopment.com/>

Uthus, O. (2021, januar 23). *Statsmeteorolog: - Når media skriver at det er orkan i*

kastene, er dette direkte feil. Nettavisen. <https://www.nettavisen.no/12-95-3424076942>

8 Vedlegg

8.1 Vedlegg 1 – Samtykkeskjemaer

Oslo, Norge / 01.05.26

Vil du delta i designprosjektet

"Geomerking – god brukeropplevelse av appen"?

Teamet mitt i emnet IN2000 ved Universitetet i Oslo jobber med å utvikle en app med navn «Geomerking». Med dette skrevet ønsker vi å informere hva prosjektet vårt har som formål, spørre deg om du vil delta i prosjektet, samt berette hva deltagelse vil innebære for deg.

Formål

Formålet med prosjektet er å undersøke hvordan vi kan videreutvikle og designe Geomerking for brukerne på beste mulig måte. Dette er en app som ønsker å gi boligkjøpere/eiere bevissthet og muligheten til å kartlegge de geologiske og meteorologiske forholdene til selvvalgte adresser i Norge. Jeg vil se på dine handlingsmønstre, interaksjoner og refleksjoner mens du utfører oppgaver i appen. Innsikten fra testen vil brukes til å videreutvikle og forbedre designet av appen slik at den i større grad støtter brukernes behov.

Deltakelse

Du blir spurt om å delta fordi du faller innenfor den relevante målgruppen, definert som "boligkjøper/eier i aldersgruppen 18-80 år bosatt i Norge". Brukertesten vil vare i maks 30 min. Det vil bli tatt skjermopptak med lyd dersom du muntlig samtykker til dette. Jeg vil også ta skriftlige notater. Dersom det tas bilder, vil de ikke kunne identifisere deg og du vil bli spurt om tillatelse til dette i forkant.

Frivillig deltagelse

Det er frivillig å delta i dette prosjektet. Du kan når som helst avslutte intervjuet eller trekke tilbake informasjon som er gitt. Du kan når som helst velge å trekke samtykket uten å måtte oppgi grunn. Dersom samtykket trekkes vil eventuelle personopplysninger som er innsamlet om deg slettes og det vil ikke innebære noen negative konsekvenser for deg at du velger å trekke ditt samtykke.

Personvern: innsamling, oppbevaring, behandling og bruk av dine opplysninger

Ingen sensitive personopplysninger (jf. Personvernforordningens artikkel 9 og 10) vil bli innsamlet. Personlige opplysninger om deg vil kun benyttes til formålene beskrevet i dette informasjonsskrivet. Vi behandler opplysningene konfidensielt og i samsvar med personvernregelverket.

Brukertesten vil kun behandles av teamet og kan ettersendes ved ønske. Dataen som oppbevares, inkludert anonymisert data, vil ikke bli publisert.

Hva skjer med innsamlet data når prosjektet avsluttes?

All rådataen blir slettet senest 05.06.26.

Rettigheter

Vi behandler opplysninger om deg basert på ditt samtykke. Så lenge du kan identifiseres i datamaterialet, har du rett til:

- innsyn i hvilke personopplysninger som er registrert om deg, og å få utlevert en kopi av opplysningene,
- å få rettet opp på personopplysninger om deg,
- å få slettet personopplysninger om deg, og
- å sende klage til Datatilsynet om behandlingen av dine personopplysninger.

Hvis du har spørsmål til studien, eller ønsker å benytte deg av dine rettigheter, ta kontakt med Veronica Corsepius Melen.

Før brukertesten begynner bes du om å samtykke i deltagelsen ved å undertegne eller gi muntlig samtykke på at du har lest og forstått informasjonen på dette arket, og ønsker å stille opp til brukertest.

Med vennlig hilsen,

Veronica Corsepius Melen
+47 45775579
veroncm@uio.no

Samtykkeerklæring

Jeg har mottatt og forstått informasjon om studentprosjektet "Geomerking – god brukeropplevelse av appen" og har fått anledning til å stille spørsmål. Jeg samtykker til:

- ☐ å delta i brukertest

Jeg samtykker til at mine opplysninger behandles frem til studentprosjektet er avsluttet.

Sted og dato

Fullt navn

Signatur

8.2 Vedlegg 2 – TeamAvtalen

Teamavtale Gruppe 20

1. Møtetidspunkt:

- o Mandager kl. 10:00-14:00
- o De 15 første minuttene er «snakkekvarter»
- o Muligens ett kortere møte i løpet av uken ved behov.

Alle medlemmer forventes å møte opp til avtalt tid. Ved 15+ minutters forsinkelse vil det forventes en enkel straff, som å ta med seg noe mat eller snacks til neste møte.

2. Tilstedeværelse og tidsbruk:

- o Alle medlemmer skal møte forberedt.
- o Fravær, sykdom eller forsinkelse skal meldes så fort som mulig, helst senest 24 timer før avtalt møte.
- o Prosjektets kjernetid er mellom kl. 06:00 og 18:00. Aktivitet utenfor dette tidsrommet behandles neste virkedag.
- o Oppgaver skal fullføres innen avtalt frist.
- o Dersom man ikke rekker en frist, skal dette kommuniseres tidlig slik at teamet kan justere planen.

3. Forventninger til den enkeltes bidrag

- o Alle skal bidra aktivt i utvikling, diskusjon og dokumentasjon.
- o Det skal være lav terskel for å be om hjelp.
- o Alle skal inkluderes i sentrale deler av prosjektet.
- o Ansvar fordeles rettferdig og tydelig.
- o Alle unngår distraksjoner i møtetid og venter med det til pausene.

4. Kommunikasjon:

- o Vi kommuniserer åpent og respektfullt.
- o Problemer tas opp tidlig.
- o Alle sier ifra og spør ved spørsmål eller uklarheter, og mottas med åpenhet av de andre.
- o Digitale kanaler brukes til korte avklaringer, men viktige diskusjoner tas fysisk.

5. Konflikthåndtering

Ved konflikter skal følgende retningslinjer følges:

1. Møtes fysisk for å diskutere situasjonen.
2. Fokuserer på problemet, ikke personen.
3. Holde dialogen saklig og løsningsorientert.
4. Dersom konflikten vedvarer, kontaktes veileder for støtte.

Signaturer:

Navn: Philip E. Fleson Signatur: Philip E. Fleson
Dato: 04.03.2026

Navn: Zusman Signatur: Zusman
Dato: 04/03/2026

Navn: Victoria Signatur: Victoria Renseth-Tiltpsen
Dato: 04/03/2026

Navn: Matilda Signatur: Matilda W. Dahl
Dato: 04/03/2026

Navn: Veronica Signatur: Veronica C. Melen
Dato: 04/03/2026

Navn: David Signatur: David Houde
Dato: 04/03/2026

8.3 Vedlegg 3 – Retrospektiv 1

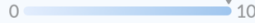
Retro 1 
Mar 26, 2026
Duration: 00:54:09

 **6**
Participants

 **77**
Responses

 **1**
Whiteboard

 **0** ✓ **0**
Action Items 

Retro Score **10.0**
0  10

Health Check Items

I regularly receive useful feedback on how good my performance is and how I can improve.

58



Looking at our recent work, I am really confident that we will deliver on expectations.

67



I am not afraid to ask "stupid" questions in my team.

75



Philip: Kunne hatt 50 mill i indeksfond, og ikke si hvilke

Victorias sticky lage video spill

Veronicas sticky Git og alt av kode

jurius's sticky tjener penger

Matilda: skrive anmeldelser

dvaids Sticky Jeg skulle vært ekspert på bygging

Icebreaker

If you could instantly become an expert in something, what would it be?

Take turns: Answer the question and pass on to the next team member.

Double click on the whiteboard to create a sticky if you want to record your answer!

Good / Bad

What are 3 recent things that went particularly well in your opinion?

effektivt arbeid

klarte å samle oss på samme belgetengde

Vi var raske med å finne ideer som alle var på lag med. Det er generelt sett lite konflikt, og vi er flinke til å løse de konfliktene/uenighetene. Alle virker motiverte for prosjektet

Maps, search, mobprog

Bekk bedpress

1. strukturen med ukes oppgaver og backlog

Kommunikasjon, team-spirit, forståelse

møtene føles nyttige

forrige møte hvor vi ble enige om hjemskjerm

2. kommunikasjon både i møter og utenfor

3. vi har vært flinke med git og github

oppmøte, alle møter opp og deltar

Check-up-en vår

What are 3 recent things that did not go so well in your opinion?

Litt uforutsigbar plan

At påskeferien kom

mobprog forsøk 1

3. det har vært litt uklarheter med selve appen (screens, funksjoner, osv)

MVP kunne vært fetere mtp utgangspunktet vårt med både ide, kompetanse og effektivitet

Jeg forsvor meg litt i dag og ikke rakk å lage lunsj

1. møtene på starten var litt overalt. Vi manglet struktur så vi endte opp med å ikke få gjort så mye (vi har blitt bedre på det)

Original mobprog, app idea cohesion, ...

2. det var veldig mye individuelt arbeid. Hadde likt om det var mer samarbeid med programmeringen og andre ting

Forsøkene våre på mobprogrammering

litt feil rekkefølge på ting, utførelse før forberedels

Aktivitetsplanlegging har vært utfordrende, felles visjon for applikasjon har vært utfordrende, case beskrivelse endring var uprofesjonelt av administrasjonen

Loved, Learned, Longed

What things did you learn lately?

coolers sine fantastiske funksjoner med fargeblindhet osv

Lærte hvordan lage gode konkrete oppgaver for backlog

Konseptet rundt debounce for å ikke gjøre for mange api kall

iconhåndtering i Figma

Shared data between screens

Git, github, tålmodighet

Hvordan bruke git

No Hidden Elements

What did you have fun with lately?

Map implementation

Progging, git og github

Figma, selv om det var mye frustrasjon også...

Bekk sin bedpress

Leste ferdig en bok etter en sikkerlig slump

Morsomt å estimere story points for backlog i den egendefinerte aktiviteten

Å se appen vår komme til liv!

at folk på gruppa vår er så flinke!

Figma

se appen bli til!

No Hidden Elements

What would you have needed lately to be able to work better?

Bedre fordeling av oppgaver, spesielt med tanke på hvilke fi hvem skal jobbe på

hakke mer strukturerte møter

Use team channels for more structured communication??

At alle deler samme ide for appen (screens, funksjoner, osv), effektive møter, flere møter (minst 2 i uken), bra og nøyte sprint plannings

Flere timer i døgnet

føle meg mer komfortabel med å røre koden hahah

hjelp med utformingen/innholdet i spørreskjema

Flere timer i døgnet?

At vi er litt mer samkjørte på hva vi jobber med kanskje?

No Hidden Elements

Hot Air Balloon Retro

Sunny skies: What can we be happy about?

alle motiverte

vi har de største implementasjonsmarerittene bak oss ("bank i bordet")

No Hidden Elements

Hot air: What is driving us?

høyt ambisjonsnivå

passion for prosjektet

liker ideen vår

noe som faktisk er nyttig og innovativt

No Hidden Elements

Sandbags: What is slowing us down?

sprint plannings, enige om hva hvem skal gjøre

usynkroniserte

vi har ikke helt kohesjon på hva appen skal være

No Hidden Elements

Storm clouds: What challenges and dangers are lurking for us?

algoritmen

spørreskjema

No Hidden Elements

35

Lean Coffee

What else do you want to talk about in our retro?

produktkravsiden - at vi lager vår egen produktkravside med referanser til vår app

No Hidden Elements

Keep, Stop, Start

What should we keep doing as a team?

keep the team-spirit

fortsette å møte opp, gjøre sitt og være engasjert/motiverte

ha møter på mandager

feedback på GitHub, fine commit og pr

gi positiv feedback kontinuerlig

No Hidden Elements

What should we stop doing as a team?

ustrukturerte møter

miskommunikasjon

No Hidden Elements

What should we start doing as a team, or at least try to?

ha et møte til i uken - statusoppdatering/samarbeidsmøter 2t

fasilitere for at alle tørr å røre gitHub

gi konstruktiv feedback dersom man tenker på noe

legge inn mer kommentarer i koden så man forstår hva tanken er

channels i teams

jobbe skjerm for skjerm

har mer struktur

8.4 Vedlegg 4 – Retrospektiv 2

Retro 2
Apr 27, 2026
Duration: 00:32:28

6
Participants

23
Responses

1
Whiteboard

4 ✓ 0
Action Items

Retro Score
9.3
-0.7 0 10

Health Check Items

Team 20

Looking at our recent work, I am really confident that we will deliver on expectations.

Your response

Agree

82 > + 15

fristen nærmer, vi har mye å gjøre, men vet hva som gjelder

Kan være ting vi ønsker å ha med men som vi ikke får tid til

Veronica Corseplus Melen's Sticky

Sy, planlegge ferien, lese seliboken i sengen min

Victoria's Sticky

Lese bok, spille nintendo, sove

dvaids Sticky

Dra hjem til Modum

Icebreaker

If you had the rest of the day off, what would you do with that time?

Take turns: Answer the question and pass on to the next team member.

Double click on the whiteboard to create a sticky if you want to record your answer!

Philip Lapetur

Matilda's Sticky

lest en bok i parken i sola

Jurius's Sticky

Spise is og fotball

Keep, Stop, Start

What should we keep doing as a team?

Delegere og hjelpe hverandre

Fredagsmøter for de som kan

Work and communicate well

ha klare sprintmål

ha mer enn ett møte i uken, har vært veldig nyttig med fredagsmøtene i tillegg

backe hverandre

Review prosessen

Sitte sammen å jobbe

jobbe effektivt med riktig mål

Hvordan vi deler ut oppgaver, basert på evne

Sprint reviews and planning + backlog refinement. Structured work.

What should we stop doing as a team?

Ta diskusjoner over melding

Lose focus about what we are talking about/planning and start talking about other points.

varre syke, neida (joda, håper alle får en sykdomsfri mai)

What should we start doing as a team, or at least try to?

Blir flinkere til å dele meninger

Kommunisere tydeligere at vi ikke kommer på planlagte møter

gi hverandre mer positiv feedback

Mer alignment ting for siste fase så alle kan bruke tiden sin effektivt

få appen på plass til slutten av denne uken

Work together (physically)

Lean Coffee

What else do you want to talk about in our retro?

Hvordan vi ligger an så langt

Innspurten, skal vi sette av mer tid til å mates? Folks tilgjengelighet fremover

37

8.5 Vedlegg 5 - Krav fra oblig 3

Funksjonelle krav:

1. Applikasjonen skal formidle oppdatert informasjon om flomfare i vassdrag.
2. Applikasjonen skal vise snøskredfare for relevante geografiske områder.
3. Applikasjonen skal visualisere kraftig vind ved hjelp av et interaktivt vindkart.
4. Applikasjonen skal vise informasjon om bølger og springflo som kan medføre skade langs kysten.
5. Applikasjonen skal vise informasjon om styrtregn som kan medføre skade på boligområder.

Ikke-funksjonelle krav:

1. Applikasjonen skal ha en responstid på maks 2 sekunder ved vanlig brukerinteraksjon under normale nettverksforhold.
2. Applikasjonen skal være kompatibelt med Android API 24 og nyere, samt støtte skjermstørrelser fra 5 til 11 tommer.
3. Applikasjonen skal automatisk oppdatere data minst hvert 2. minutt.
4. Applikasjonen skal ha en modulær arkitektur med tydelig separasjon mellom UI, logikk og datatilgang (MVVM-arkitektur).

8.6 Vedlegg 6 - Krav gruppert etter domene

Funksjonelle krav	Ikke-funksjonelle krav
<i>Produktkrav</i>	
	Applikasjonen skal være kompatibel med Android API 24 og nyere, og støtte skjermstørrelser fra kompakt til utvidet.
Applikasjonen skal ikke krasje ved manglende internett eller mislykkede API-kall, men varsle brukeren med en tydelig feilmelding	
	Applikasjonen skal følge MVVM-arkitektur med tydelig separasjon mellom UI, domenelogikk og datatilgang
	API-kall skal gjøres asynkront og ikke blokkere brukergrensesnittet.

	Viktig tilstand som valgt lokasjon skal ikke forsvinne ved rotasjon, avbrudd eller navigasjon mellom skjermer.
Koden skal skrives på engelsk	
<i>Krav fra casebeskrivelse</i>	
Applikasjonen skal vise historisk klimadata (temperatur, nedbør, snø, vind og solskinn) for en valgt lokasjon, basert på aggregerte tidsserier fra nærmeste meteorologiske målestasjoner via Frost API v0	
Applikasjonen skal vise flom- og skredfaresoner på kart ved hjelp av WMS-kartlag fra NVEs kartkatalog	
<i>Teamets løsning</i>	
Applikasjonen skal la bruker søke opp en adresse og få opp klimainformasjon og risikovurdering for den valgte lokasjonen	
	Applikasjonen skal bruke Google Maps som kartleverandør.
Applikasjonen skal beregne og vise en GeoScore for valgt lokasjon, basert på relevante faretyper som flom og skred.	
Applikasjonen skal gi konkrete anbefalinger om klimatilpasningstiltak for bolig tilpasset for en valgt lokasjon.	
Applikasjonen skal inkludere en tydelig ansvarsfraskrivelse om at GeoScoren er et estimat og ikke erstatter profesjonell rådgiving	
Når en bruker har valgt en lokasjon, skal lokasjonen merkes med en pin på kart	
Applikasjonen skal la bruker velge lokasjon ved å trykke på kartet	

Applikasjonen skal la brukere lagre og slette valgt lokasjon med tilhørende generert klimadata	
Applikasjonen skal tilby en navigasjonsmeny som lar brukeren navigere mellom hovedsidene hjem, kart og lagret.	
Applikasjonen skal generere en tekstlig beskrivelse av beregnet GeoScore for en gitt lokasjon ved bruk av ChatGPT av OpenAI.	
<i>Universell utforming og WCAG-kravene</i>	
	Applikasjonen skal følge WCAG 2.1 suksesskriterium 2.1.3 (Statusbeskjeder) nivå AA
	Applikasjonen skal oppnå et kontrastnivå på 4,5:1 på tekst som følge av WCAG 2.1 suksesskriterium 1.4.3 (Kontrast) nivå AA
Applikasjonen skal tilby tekstlig alternativer for viktig ikke-tekstlig innhold	Applikasjonen skal ta hensyn til WCAG 2.1 suksesskriterium 1.1.1 (Tekstalternativ) nivå A
Bruker skal kunne velge visningsretning (stående eller liggende) i applikasjonen	Applikasjonen skal ta hensyn til WCAG 2.1 suksesskriterium 1.3.4 (Visningsretning) nivå AA
	Applikasjonen skal ta hensyn til WCAG 2.1 suksesskriterium 2.4.1 (Hoppe over blokker) samt 2.4.3 (Fokusrekkefølge) nivå A

	Applikasjonen skal ta hensyn til WCAG 2.1 suksesskriterium 4.1.2 (Navn, rolle, verdi) nivå A for å støtte skjermleserbrukere.
	Applikasjonen skal ta hensyn til WCAG 2.1 suksesskriterium 3.3.2 (Ledertekster eller instruksjoner) nivå A

8.7 Vedlegg 7 - Google Maps Github Issue og Version 8.3.0 Release

WmsUrlTileProvider(EPSG:3857) class that is a subclass of UrlTileProvider and a WmsTileOverlay composable that wraps TileOverlay and uses said provider. #880

Closed #884



munw37 opened on Mar 27

Is your feature request related to a problem? Please describe.
While doing a student-app-project mandatory APIs were on WMS EPSG3857 format. Seeing the general ease of use of the GoogleMap-composable and some previous knowledge I had about WMTS I figured I could just convert as I learned more about the WMS-format. Checking the network requests made by the MapLibre example provided to us I saw that it in fact did query the WMS-API in a square tile-based manner too.

Describe the solution you'd like
I have written code for:
A URL-tile provider subclass that takes a urlFormatter-lambda with arguments for the bounding-box (xMin,yMin,xMax,yMax) that returns a string, so that the programmer can dictate the formatting of the URLs sent from the tile provider to the tile overlay. To make it efficient the provider uses bit shifting for powers of 2 used in BBOX calculations. (IE finding the amount of tiles across for a given zoom-level)
Another efficiency is that if given bounds of the API the overlay is showing, the provider will return null in place of URLs pointing to tiles outside of the APIs bounds foregoing a "doomed" network request.

I also wrote a TileOverlay-wrapper that sets up its own TileProvider, with the only requirement of a urlFormatter-lambda it can pass down to the provider.

Assignees
No one assigned

Labels
priority: p2 type: feature request

Type
No type

Projects
No projects

Milestone
No milestone

Relationships
None yet

8.3.0 (2026-04-09)

Features

- add WMS tile overlay support to maps-compose-utils (#884) (ca66e98)
- expose rememberComposeBitmapDescriptor as public experimental API (#867) (3456db0)

8.8 Vedlegg 8 – Skjermbilder av appen



Vedlegg 10 intervjuguidet

Brukertest plan

Praktisk

Ordstyrer: Veronica

Noterer, tidpårpasser og fotoansvarlig: Veronica

Tid: 20-30 min

Hvordan

- Observasjon og thinkaloud
- Ta skjermopptak med lyd på android tlf
- Noterer på macen underveis, evt papir/bok
- Appen + Litt Figma

Ha med

- Utskrift av plan
- Utskrift av samtykkeskjema

Brukertest guide

Introduksjon - 2 min

Samtykkeskjema

→ Signere det

Boligkjøp - status

- Eier/leier:
- Når:
- Tidsperspektivet:

Prompt

Valgt case 3 med naturhendseler, utviklet appen som begynner å ta litt form, og nå ønskes å testes. Viktig å poengtere at det er ikke deg men **prototypen vi ønsker å teste**.

Våg å si din meninger og tanker, de tas i mot med takknemlighet og **ikke som kritikk**.

Valgt å teste med deg siden du kan relatere til vår målgruppe som er boligkjøpere(eiere) i aller aldersgrupper i Norge.

Vi har noen **oppgaver** du skal utføre mens jeg observerer - og da ønsker vi at du **tenker høyt** mens du navigerer så vi ****blir **med på din forståelse** av appen, i tillegg til **spørsmål** vi ønsker svar på fra deg.

Vil primært teste appen, men også litt i Figma

Hoveddel - 15min (1min per punkt)

Generelt

- navigasjon
 - intuitiv nav bar uten tekst?

HomeScreen

- tekst
 - overskrift
 - “grunnforhold”

HomeScreen_report

- kom rapporten brått på?
 - alt 1, 2, 3 eller 4
- Skala
- Sammendragkortet
 - Lagre-knapp
 - Slidere - savner du noe som illustrerer skalaen?
- Detalj kortene
 - Detalj - bokstavene
 - Tekst
- Vis i kart i detalj kort eller i sammendragkortet?

HomeScreen_historical

- Grafene:
 - Hva tolker du dem at de viser?
 - Enhetene?

MapScreen

- hvilke kartlag kunne du tenkt deg å ha her?
- bottom-sheet
- Pin
- kunne du tenkt deg å få muligheten til å lage risiko-rapport av pin-en her?

SavedScreen

- Un-lagre
- sammenligne kun med bokstav eller alle risikoene?

Noe du savner i appen?

- laste ned?

HUSK å ta bilde!!!

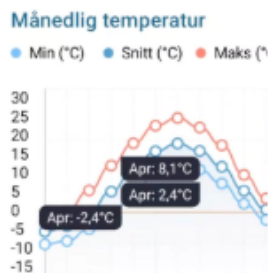
Brukertest funn m/action points

Fikse/finne ut av

- HomeScreen_searching
 - "overflødig repetisjon av tekst" → fjern repetert tekst
 - søker Tjuvholmen og får opp alle andre enn Oslo før han skrev Oslo → endre på søkingen, hvis det går an?
 - "siste søk, den var fin" → beholde siste søkt, kalle den for "nylig søkt"
- grafene @Matilda Dahl
 - → legge til større loading-feedback på historisk data, grafene



- → tilpasse y-aksen til verdiene for punktene
- skyve bortover funksjon? → undersøke om dette er stress
- Vind
 - viser ikke hele teksten → edit vindgrafen så teksten synes/gir mening
- "At den oppgir interpolerte data mellom månedene er tull" → ha "snapping"
- svart boks, synes ikke når fingeren er oppå, forvinner fort når fjerner finger → forlenge den tiden?



- Rekkefølgen
 1. Temp
 2. Vind
 3. Sol
 4. Nedbør
 5. Snø

→ endre rekkefølgen → done!